# 貪婪法

每一步選擇眼前最有利的候選，能否保證整體最優？關鍵是證明這個選擇仍能延伸成最優解。

本章依序研究 **活動選擇 → Huffman 編碼 → 擬陣與工作排程**，從具體交換論證走向可重用的結構條件。

前置知識為排序、優先佇列、樹、集合與歸納證明。學習目標是分清局部選擇、可行性與最優性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 從選擇規則走到正確性

| 每次設計都要回答 | 所需證據 |
| --- | --- |
| 什麼是合法解？ | 固定相容性、前綴碼或獨立集合的限制。 |
| 眼前選哪一項？ | 清楚定義候選、排序與平手處理。 |
| 為何不失去最優解？ | 交換、縮約或擬陣交換性質。 |
| 改了目標還成立嗎？ | 用反例檢查加權或限制改變後的規則。 |

活動選擇、Huffman 與擬陣各有自己的證明，不可只憑「看起來合理」套用同一條規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 用最早結束保留後續選擇

一間教室、固定活動時間、每個活動價值相同。目標是選最多個互不衝突的活動。

貪心規則是每次選「與已選活動相容、且最早結束」的活動。正確性來自可交換的最優解，不是因為例子剛好成功。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 端點相接算相容

活動 i 使用半開區間 $[s_i, f_i)$，假設 $s_i < f_i$。

兩活動相容當且僅當 $f_i \le s_j$ 或 $f_j \le s_i$。已選活動最後在 L 結束時，候選的判斷式是 $s_i \ge L$。

[0,3) 與 [3,5) 相容；[0,3) 與 [2,5) 衝突。若場地需要清場時間 δ，條件須改成 $s_i \ge L+\delta$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

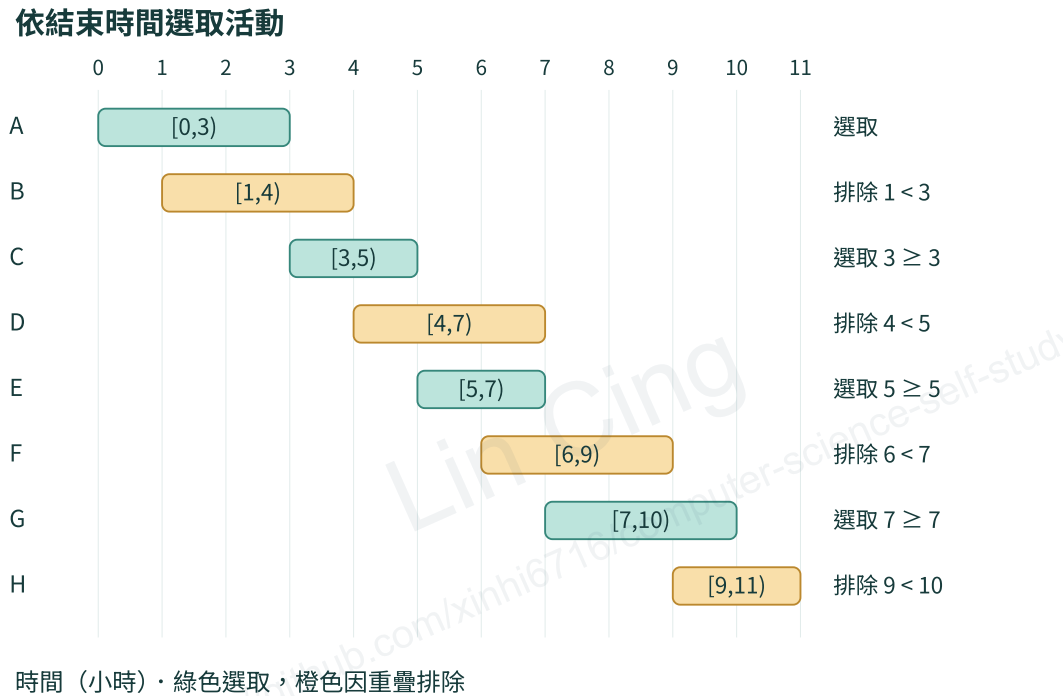
# 先按結束時間排序再掃描

```
selected = []
last_finish = −∞
for (id, start, finish) in sort_by_finish(activities):
    if start >= last_finish:
        append id to selected
        last_finish = finish
return selected
```

同結束時間可任意固定順序。本例採 ID 順序。空輸入回傳空集合；拒絕活動時不更新 last_finish。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 八個活動的時間線




時間以小時計。輸入 A[0,3)、B[1,4)、C[3,5)、D[4,7)、E[5,7)、F[6,9)、G[7,10)、H[9,11)。綠色選取 A、C、E、G；橙色標出與當時最後活動衝突者。

# 每一步只比較最後結束時間

| 候選 | 掃描前 L | 判斷 | 動作 | 掃描後 L |
|---|---|---|---|---|
| A [0,3) | $-\infty$ | $0 \geq -\infty$ | 選 A | 3 |
| B [1,4) | 3 | $1 < 3$ | 排除 B | 3 |
| C [3,5) | 3 | $3 \geq 3$ | 選 C | 5 |
| D [4,7) | 5 | $4 < 5$ | 排除 D | 5 |
| E [5,7) | 5 | $5 \geq 5$ | 選 E | 7 |
| F [6,9) | 7 | $6 < 7$ | 排除 F | 7 |
| G [7,10) | 7 | $7 \geq 7$ | 選 G | 10 |
| H [9,11) | 10 | $9 < 10$ | 排除 H | 10 |

輸出 4 個活動。最大個數不保證答案唯一，例如 A、C、E、H 也是 4 個。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 最早開始與最短時間都可能失敗

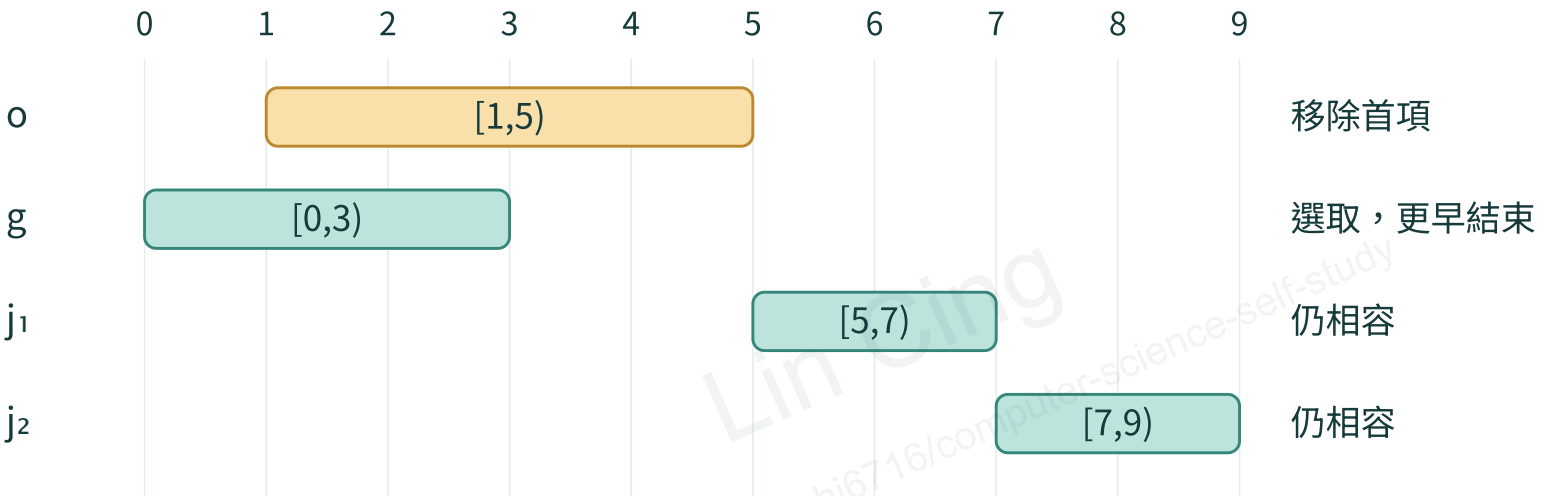
| 規則 | 反例區間 | 規則得到 | 更佳答案 |
| --- | --- | --- | --- |
| 最早開始 | X[0,8)、A[1,3)、B[3,5)、C[5,7) | X，1 個 | A、B、C，3 個 |
| 最短時間 | A[0,4)、X[3,5)、B[4,9) | X，1 個 | A、B，2 個 |

最短時間法在這裡指反覆從仍相容的活動中選最短者。X 長度 2，嚴格短於 A、B，但跨越兩者接點。最早開始不能用來推論最早結束也失敗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 交換第一個活動而不損失數量

## 以 g 替換 o，後續活動完全保留




$f_g = 3 \leq f_o = 5 \leq s_{j_1} = 5$

考慮任意非空剩餘問題，所有候選均在目前邊界 L 之後開始。令 g 為候選中最早結束者，O 為一個最大可行解，o 為 O 的第一個活動。因 g 最早結束，$f_g \leq f_o$。

# 交換後仍可行，且仍最優

若 g=o，O 已包含貪心選擇。否則令 $O' = (O - \{o\}) \cup \{g\}$。

1. g 的開始時間 ≥ L，所以不衝突於已固定的前綴。
2. O 中其餘活動 j 都在 o 之後，故 $s_j \geq f_o \geq f_g$，換成 g 後仍相容。
3. g 不會原本就在 O 的後段，因正長度與相容性會推出 $f_g > f_o$，與 $f_g \leq f_o$ 矛盾。
4. 所以 $|O'|=|O|$。$O'$ 仍最優，且以 g 開頭。

結論是「存在一個包含 g 的最優解」，不是「每個最優解都包含 g」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 剩餘問題與歸納完成整體證明

選 g 後只留下 $R=\{i \mid s_i \geq f_g\}$。沒有活動能排在 g 前面，因所有候選的結束時間 $\geq f_g > s_g$。

若包含 g 的最優解，其後綴不是 R 的最優解，就能用更大的後綴替換而得到更大的原解，矛盾。因此 $OPT(S)=1+OPT(R)$。

空集合是歸納基底。交換論證保證第一步安全，對更小的 R 重複同一論證，得到整個貪心解最優。單有最優子結構不足以證明貪心選擇安全。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 時間成本與證明適用範圍

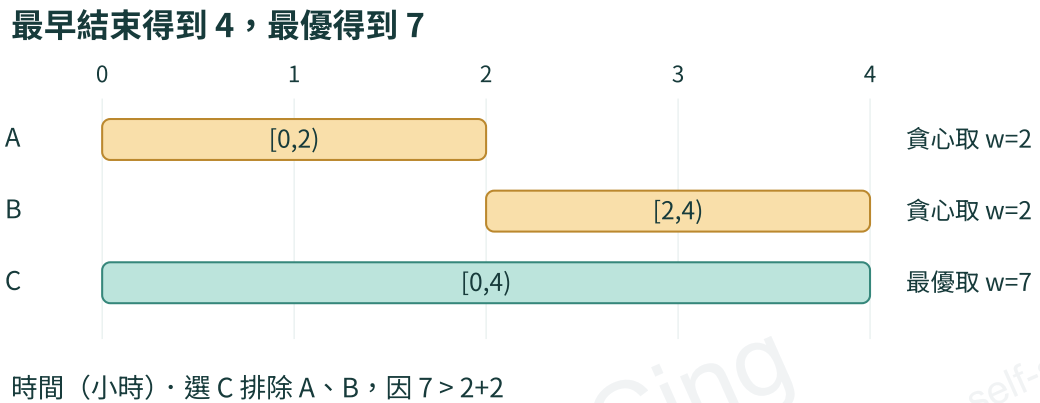
排序 $O(n \log n)$，掃描 $\Theta(n)$，總時間 $O(n \log n)$。若輸入已按結束時間排序，則總時間 $\Theta(n)$。

除排序與輸出外，掃描只需 $O(1)$ 額外空間；輸出至多 $n$ 個 ID。

本證明假設單一資源、固定正長度區間、最大化個數。由右往左每次選最晚開始的相容活動，也可作對稱交換證明；不能把它誤寫成由左往右最早開始。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 加入權重後交換不再保值




A[0,2) 權重 2，B[2,4) 權重 2，C[0,4) 權重 7。最早結束選 A、B，總值 4；最優選 C，總值 7。

交換保住「個數」卻不能保住「總值」。這推翻原規則於加權版本的正確性，不代表已證明所有可能的貪心規則均失敗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 加權區間用取或不取的動態規劃

依結束時間排序，$p(j)$=最大的 $i<j$ 且 $f_i \le s_j$；不存在則 $p(j)=0$。令 $M(j)$ 為前 $j$ 個活動的最大總權重。

$M(0)=0$，$M(j)=\max(M(j-1), w_j+M(p(j)))$。

不取 $j$ 時答案在前 $j-1$ 個中；取 $j$ 時，其前綴只能來自前 $p(j)$ 個。兩種情況涵蓋所有最優解。

排序 A、B、C，$p=(0,1,0)$，$M=(0,2,4,7)$。回溯比較兩支可還原 {C}。二分搜尋求 $p$，總時間 $O(n \log n)$，DP 與回溯 $O(n)$，空間 $O(n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 把頻率轉成最短的總位元數

Huffman 解有限字母表的二元、逐符號、無前綴碼最佳化。輸入是非負權重 $w_c$，輸出是每個符號的碼字。主要證明先設 $n \geq 2$ 且權重正值，最後擴充邊界。

| 符號 | a | b | c | d | e | f |
|---|---|---|---|---|---|---|
| 頻率，千次 | 45 | 13 | 12 | 16 | 9 | 5 |

六種符號的定長碼需每次 3 bits。本例 Huffman 平均 2.24 bits，共 224 千 bits，未計碼表與框架。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 無前綴性讓串接可即時解碼

無前綴 prefix-free 意指任一碼字都不是另一個碼字的前綴。二元樹中只有葉子代表符號，左邊標0，右邊標1。

例如 $\{0, 10, 110, 111\}$ 無前綴；$\{0, 01\}$ 不成立。讀到葉子就輸出符號並回到根，不需要分隔符。

互異碼字仍可能無法唯一解碼，例如 $\{0, 01, 10\}$ 中，位元串010可分成0|10或01|0。無前綴性是充分條件，但不是唯一可解碼的必要條件。

這裡的最佳化範圍是逐符號無前綴碼，不宣稱可勝過使用符號間相依性的區塊壓縮。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Kraft 不等式限制的是碼長

對有限二元無前綴碼，碼長 $\ell_i$ 是非負整數，必有

$$\sum_i 2^{-\ell_i} \le 1.$$

深度 $\ell$ 的節點對應 $[0,1)$ 中長度 $2^{-\ell}$ 的二進位區間。無前綴使這些區間互不重疊，因此總長不超過1。

反向也成立。將碼長由短到長排列，依序取尚未使用的最左 dyadic 區間。先前較短區間占滿深度 $\ell$ 的整數格；總長限制保證仍有一格可取，得到所需無前綴碼。

例如碼長1,2,3,3的和為1；1,1,2的和為1.25，故不可能。$n \ge 2$ 時碼長至少1；空碼長0只容許單一符號。這個判準不能直接用在任意僅碼字互異的碼。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 加權外部路徑長就是成本

對樹 $T$，葉深度 $d_T(c)$ 等於碼長。目標是

$$B(T) = \sum_c w_c d_T(c).$$

若 $w_c$ 為出現次數，$B$ 的單位是 bits；若為機率，$B$ 為 bits/符號。正權重下，單子女內部節點可刪除，受影響葉子深度下降且成本嚴格下降，所以最優樹必為滿二元樹。

每個內部節點的權重是其葉子權重總和。每個葉權重被其所有祖先各算一次，因此

$$B(T) = \sum_{v \text{ internal}} w(v) = \text{所有合併權重之和}.$$

這是兩種獨立核算成本的方法；根權重則只有 $\sum_c w_c$，不等於總成本。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 每輪取出兩棵最輕的樹

佇列元素是整棵子樹，key為子樹權重。每次合併都把兩個元素變成一個，故恰好 $n-1$ 輪。

```
Q ← min-priority queue of symbol leaves
while |Q| > 1:
    x ← extract-min(Q)
    y ← extract-min(Q)
    z ← Node(weight=x.weight+y.weight, left=x, right=y)
    insert(Q,z)
return extract-min(Q)
```

不變量是佇列中的子樹葉集合互斥且聯集為原字母表，佇列權重總和保持初始總權重 $W$；本例 $W=100$。新節點必須放回佇列，不能只掃原始頻率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 五輪佇列，逐輪可追查

表中佇列僅為顯示而排序，heap內部不必完全排序。$z_i$ 是第 $i$ 輪新節點。

| 輪 | 取出 | 插入 | 合併後佇列，依權重排序 |
|---|---|---|---|
| 0 | | | f5,e9,c12,b13,d16,a45 |
| 1 | f5,e9 | $z_1$14 | c12,b13,$z_1$14,d16,a45 |
| 2 | c12,b13 | $z_2$25 | $z_1$14,d16,$z_2$25,a45 |
| 3 | $z_1$14,d16 | $z_3$30 | $z_2$25,$z_3$30,a45 |
| 4 | $z_2$25,$z_3$30 | $z_4$55 | a45,$z_4$55 |
| 5 | a45,$z_4$55 | $z_5$100 | $z_5$100 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 五輪佇列，逐輪可追查 （續）

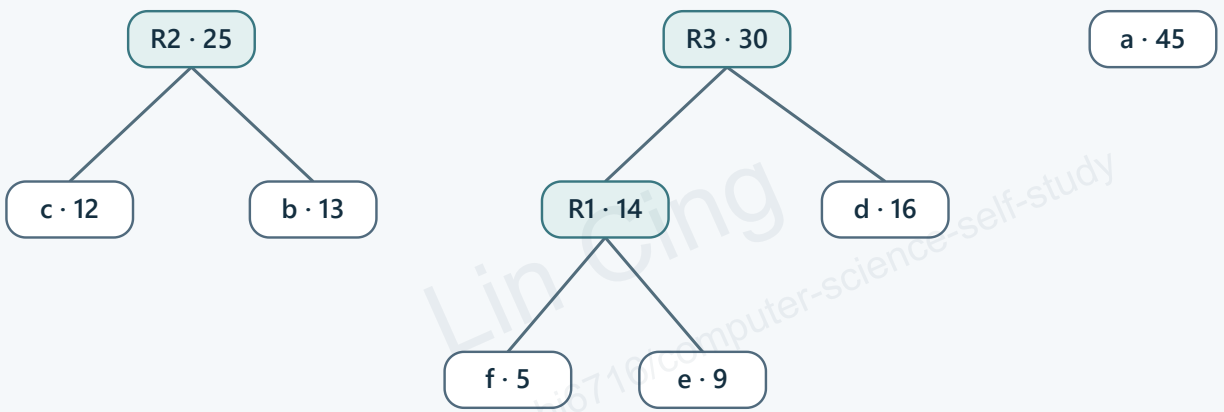



第三輪合併後，佇列只有三棵樹，權重25、30、45。下一輪要合併25與30，不是將新節點30直接接到45。

# 中途的森林也是演算法狀態

### 第3輪・三棵樹仍在佇列

R 節點標出合併輪次與權重；下一輪合併25與30



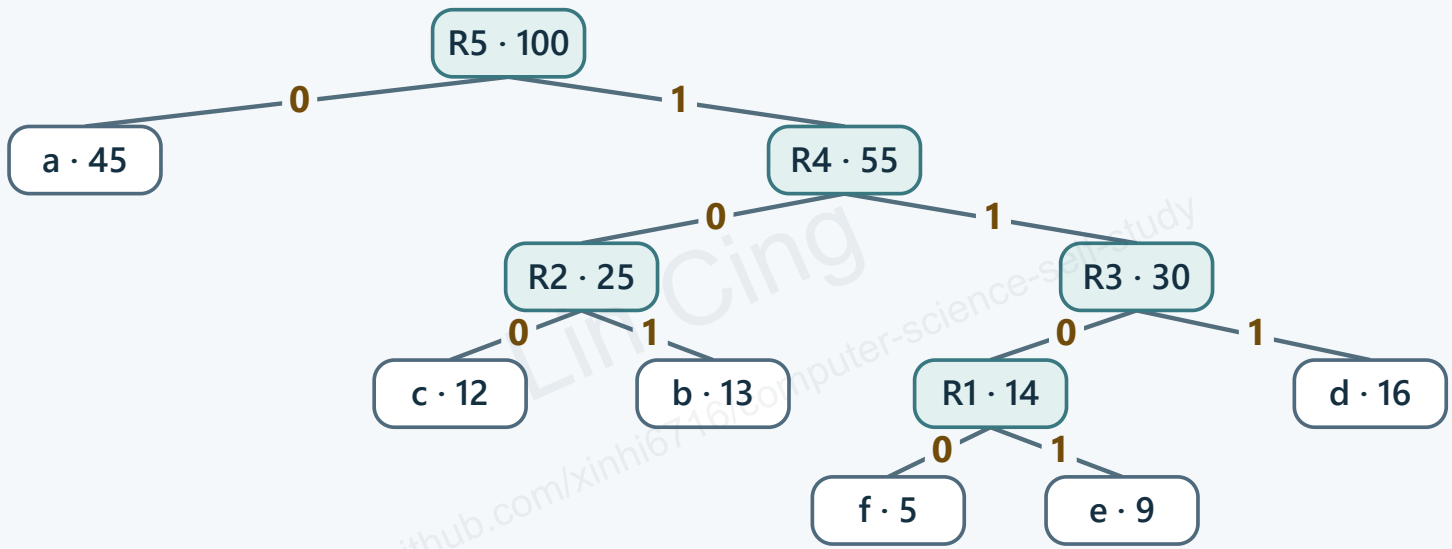

25 + 30 + 45 = 100

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 由根讀碼，再用兩種方式核算

## 最終樹 · 根權重 100 · 成本 224

左 0、右 1；R 表示合併輪次；頻率單位為千次




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 由根讀碼，再用兩種方式核算 （續）

| 符號 | a | b | c | d | e | f |
|---|---|---|---|---|---|---|
| 碼字 | 0 | 101 | 100 | 111 | 1101 | 1100 |
| 深度 | 1 | 3 | 3 | 3 | 4 | 4 |

合併權重14、25、30是之後成本的已定部分。在子樹上方加入新父節點後，每個後代葉的深度增加 1，新增成本剛好是該子樹權重。

$B = 45 + 39 + 36 + 48 + 36 + 20 = 224$ 千 bits。

合併核算為 $14 + 25 + 30 + 55 + 100 = 224$。根權重100千次。Kraft和 $1/2 + 3/8 + 2/16 = 1$。例如abc編成0101100。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 交換論證先證明有最優樹接受貪心選擇

設 $x, y$ 是兩個最小權重。取一棵最優滿二元樹，選最大深度 $D$ 的兄弟葉位置 $a, b$。最大深度葉的兄弟必也是葉，否則會有更深的葉。

先把 $x$ 與 $a$ 交換。若 $x$ 本來在 $b$，先交換這對兄弟位置，這不改成本。交換成本差為

$$B(T_{new}) - B(T) = (w_a - w_x)(d_T(x) - D) \leq 0.$$

接著把 $y$ 與位置 $b$ 的葉交換。$x$ 已固定在 $a$，剩下的 $y$ 是未固定葉中的最小權重，因此相同不等式成立。若符號已在目標位置就略過交換。

兩次都不增成本，原樹已最優，故新樹仍最優。結論是「存在」最優樹使$x, y$為最深兄弟，不是所有最優樹都必須長成此形。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 收縮與歸納把局部選擇接到全域最優

把兄弟 $x, y$ 收縮成葉 $z$，令 $w_z = w_x + w_y$。其他深度不變，兩葉各少走一條邊。

$$B(T) = B(T') + w_x + w_y.$$

若最優樹收縮後的 $T'$ 不最優，用較便宜的縮小樹替代，再展開 $z$，增加的仍是固定 $w_x + w_y$，就會比原最優樹更便宜，矛盾。

歸納以一個葉子成本0為基底。交換引理保證某個最優解含本輪最小兄弟；歸納假設保證 Huffman 解好 $n - 1$ 個符號的收縮問題；展開恢復固定成本，所以得到原問題最優解。

局部「本次合併最便宜」本身不構成證明。真正的保證是交換不增成本與收縮後保留最優子結構。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 同權重、單一符號與零權重

同權重可任意打破平手。左/右互換也不改碼長成本；要重現同一碼表，可用固定的建立序號當第二key。

單一符號的樹模型用空碼 $\epsilon$，成本0。實作若強制非空碼可指定0，成本 $w$；這是不同的碼長限制。空碼解碼需要外部給定符號個數。空字母表則直接回傳空結果。

零權重若仍在字母表內，必須分配碼字。Huffman與不增成本證明仍適用，但刪單子女可能只是不增成本，所以不能宣稱每棵最優樹都滿。例如a權重1、b權重0，用00與01成本2，比0與1成本1差；全零時任何合法碼成本皆0。

權重必須非負且有限。若總權重0，不能除以總權重計算平均碼長。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 複雜度與可驗證的交付

二元 min-heap 建立需 $O(n)$。每輪兩次extract-min與一次insert各 $O(\log n)$，共 $n-1$ 輪，所以建樹時間 $O(n \log n)$、節點空間 $O(n)$，共有 $2n-1$ 個節點。

若權重預先排序，可用原葉佇列與合併節點佇列，在兩個佇首中取最小值，建樹 $O(n)$。排序成本另計。

這些是建樹成本。若直接儲存所有碼字字串，若為每個符號記錄一個碼字，輸出時間與空間還需 $\Theta(n + \sum_c \ell_c)$，最壞可達 $\Theta(n^2)$；編碼訊息還需處理實際輸出的位元。

逐一檢查碼字互非前綴，並計算葉深成本 45+39+36+48+36+20=224 與合併成本 14+25+30+55+100=224。最優性由交換與縮約證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 何時「選最重」能保證最佳？

活動選擇有專用的最早結束規則。Matroid 則用集合結構保證任意非負權重的排序貪心。

本單元把「可行解」寫成獨立集合，逐步證明交換，再把單位作業的逾期罰款改寫為保住的權重。

閱讀主線為 CLRS §16.4–16.5。無向圖森林只用來追蹤集合操作，MST 演算法與 cut property 留第 8 章。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 有限 ground set 與兩個公理

令 M=(E,$\mathcal{I}$)，E 是有限 ground set，$\mathcal{I}$⊆2^E，且 ∅∈$\mathcal{I}$。

- 遺傳性。A∈$\mathcal{I}$ 且 $B \subseteq A$，則 B∈$\mathcal{I}$。
- 交換性。A,B∈$\mathcal{I}$ 且 |A|<|B|，則存在 x∈B\A，使 A∪{x}∈$\mathcal{I}$。

滿足這些條件才是 matroid。只有遺傳性的結構稱獨立系統。交換公理量化的是每一對獨立集合，不是只檢查一組成功例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 極大與最大不能混用

極大獨立集合無法再加入元素。最大獨立集合有最多元素。

在 matroid 中，若極大集合 A 比獨立集合 B 小，交換性立即給 A 一個可加入元素，矛盾。因此所有極大獨立集合等大，稱為基底，其大小為 rank r。

Uniform matroid $U_{2,3}$ 有 $E=\{a,b,c\}$，$\mathcal{I}=\{X\subseteq E:|X|\leq 2\}$。三個二元素集合都是基底，單元素集合都不是極大。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

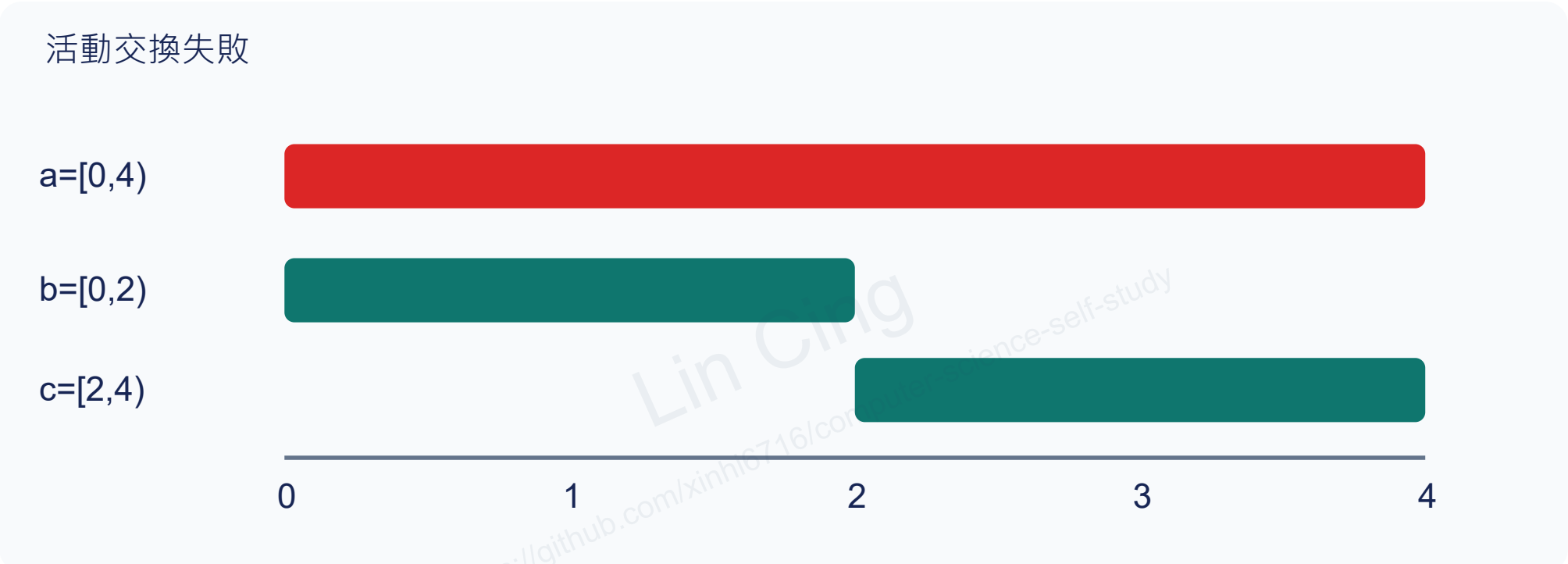
# 活動選擇的獨立系統不是 matroid

採半開區間。a=[0,4)、b=[0,2)、c=[2,4)。獨立集合是兩兩不衝突的活動集合。

A={a} 與 B={b,c} 都獨立，|A|=1<2=|B|。但 A∪{b} 和 A∪{c} 都衝突，交換性失敗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 活動選擇的獨立系統不是 matroid （續）




遺傳性仍成立。最早結束的專用證明不表示活動獨立系統是 matroid。

# 加權貪心與負權重的界線

目標為 $\max \Sigma_{x \in A} w(x)$，條件 $A \in \mathcal{I}$。先按 $w$ 遞減排列，$A \leftarrow \varnothing$。依序掃描 $x$，若 $A \cup \{x\} \in \mathcal{I}$ 就加入。

當 $w \geq 0$，這個版本求得最大權重獨立集合，零權重可加入，輸出一個基底。排序 $O(n \log n)$，$n$ 次獨立性查詢，每次 $T_{ind}$，總計 $O(n \log n + nT_{ind})$。

允許負權重且可自由選子集時，跳過所有 $w(x)<0$。例 $U_{2,2}$，權重 $5,-7$，強制加滿得 $-2$，而只取第一個得 5。若題目要求基底，則不能跳過負權重，原排序規則仍求最大權重基底。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 交換證明 1｜先準備一個最佳基底

先考慮非負權重。任一最佳獨立集合可擴成基底而不減權重，因此存在最佳基底 $O$。

維持不變量「某個最佳基底 $O$ 包含目前貪心集合 $G$」。初始 $G=\varnothing$ 成立。下一個接受元素為 $g$。若 $g\in O$，不需改 $O$。

若 $g\notin O$，先令 $I_0=G\cup\{g\}$，再設 $H=I_0$。因 $|H|<|O|$ 時可用交換性，反覆從 $O\setminus H$ 加入元素直到 $|H|=|O|=r$。每步都保留獨立性與 $G\cup\{g\}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 交換證明 2｜替換一項且不減權重

完成後 $H \subseteq O \cup \{g\}$、$|H|=|O|$ 且 $G \cup \{g\} \subseteq H$，故 $H=(O\{y\}) \cup \{g\}$，其中 $y \in O \setminus G$。

因 $G \cup \{y\} \subseteq O$，$y$ 在目前 $G$ 下可行。$y$ 不可能已在更早被拒絕，否則當時的較小前綴加 $y$ 不獨立，與遺傳性矛盾。因此排序保證 $w(g) \geq w(y)$。

$w(H)=w(O)-w(y)+w(g) \geq w(O)$。$O$ 已最佳，故 $H$ 也是最佳，並包含新的 $G$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 交換證明 2｜替換一項且不減權重 （續）

以一次交換保留貪心前綴

$G \subseteq O$ → $I_0 = G \cup \{g\}$ → $H = O - \{y\} + \{g\}$

$I_0 \subseteq H \subseteq O \cup \{g\}$, $|H| = |O| = r$

反覆加入 O 中元素，保持獨立

$y \notin G$, $w(g) \geq w(y)$, $w(H) \geq w(O)$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

掃描結束時 G 是基底，包含它的最佳基底只能等於 G。負權重的自由子集版在非負元素的限制 matroid 上套用同一證明。

# Graphic matroid｜邊集合的可行性是無環

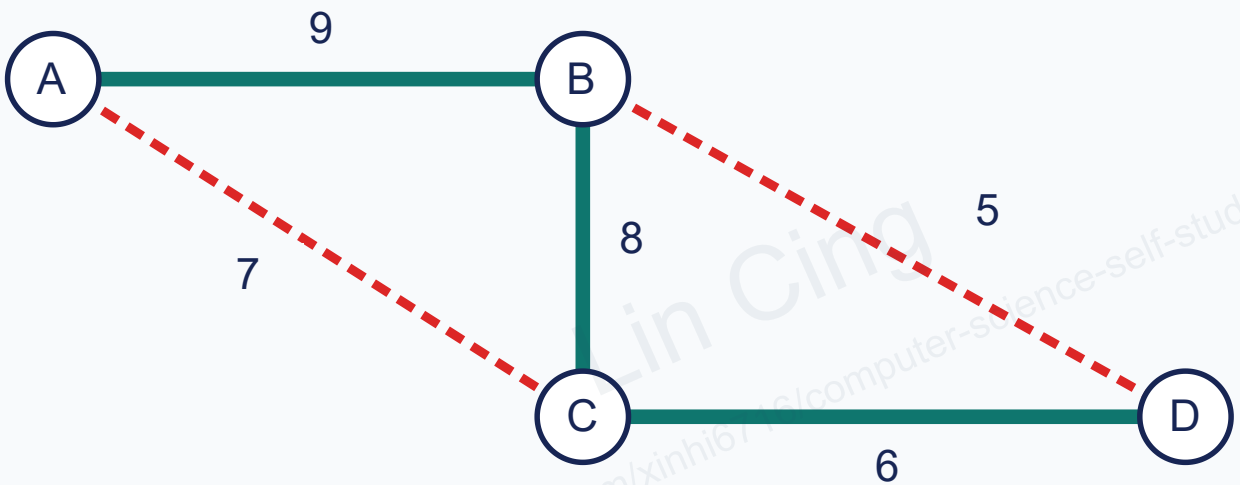
給無向圖 V={A,B,C,D}，E={AB,BC,AC,CD,BD}。$\mathcal{I}$ 是不含無向環的邊集合。

刪除森林的邊仍是森林。若森林 F 比森林 H 邊少，H 必有一條邊跨越 F 的兩個連通分量，否則 H 在每個 F 分量內最多只能有 |V_i|−1 條邊，合計不超過 |F|。跨分量邊可加入 F 而不成環，故有交換性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Graphic matroid｜邊集合的可行性是無環 (續)

最大權重森林的接受與拒絕




AB, BC, CD

9 + 8 + 6 = 23

AC, BD rejected

本例是最大權重森林，不是最小生成樹的逐步教學。

# Graphic matroid｜追蹤每次接受與拒絕

沿用上圖，權重 AB=9、BC=8、AC=7、CD=6、BD=5。

| 掃描邊 | 判定 | 掃描後的 G | 總權重 |
|---|---|---|---|
| AB | 接受，連接 A 與 B | {AB} | 9 |
| BC | 接受，連接 C | {AB,BC} | 17 |
| AC | 拒絕，形成 A–B–C–A 環 | {AB,BC} | 17 |
| CD | 接受，連接 D | {AB,BC,CD} | 23 |
| BD | 拒絕，形成 B–C–D–B 環 | {AB,BC,CD} | 23 |

三條接受邊形成基底，權重 23。此處的查詢只問加邊後是否仍為森林。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 單位作業｜最小罰款等於最大保住權重

所有作業在時間 0 可用，單一處理器，每件恰需 1 單位時間，不搶先。deadline d_i 為 1 到 n 的整數。完成時間 $C_i \le d_i$ 免罰，否則支付非負 penalty $p_i$。每件作業最終都要執行。

獨立集合 A 是「能全部準時完成的作業子集」。令 $N_t(A) = |\{i \in A : d_i \le t\}|$。

$\min \sum_{i \text{ late}} p_i = \sum_i p_i - \max_{A \in \mathcal{I}} \sum_{i \in A} p_i$。

| 作業 | $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ | $a_7$ |
|---|---|---|---|---|---|---|---|
| $d_i$ | 4 | 2 | 4 | 3 | 1 | 4 | 6 |
| $p_i$ | 70 | 60 | 50 | 40 | 30 | 20 | 10 |

數據採 CLRS 圖 16.7，總 penalty 為 280。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 可行性判準與排程方法

$A \in \mathcal{I}$ 當且僅當每個整數 $t \in [0, n]$ 都滿足 $N_t(A) \leq t$，也等價於依 deadline 遞增排序後全部準時。

必要性。若 $N_t > t$，必須在前 t 格完成超過 t 件單位作業，不可能。

充分性。把 deadline 排成 $d_{(1)} \leq \cdots \leq d_{(k)}$。若第 j 件逾期，即 $d_{(j)} < j$，則 $N_{d_{(j)}} \geq j > d_{(j)}$，矛盾。故第 j 件在時間 j 完成且 $j \leq d_{(j)}$。

遺傳性由 $N_t(B) \leq N_t(A)$ 得到。可行性與 penalty 無關，penalty 只決定貪心掃描順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 作業交換性｜每一個時間前綴都要成立

設 A,B 可行且 |A|<|B|。取 k=max{t:$N_t(B) \leq N_t(A)$}。因 N₀(A)=N₀(B)=0，k 存在。因 N_n(B)>N_n(A)，k<n。

在 k 處差值 ≤0，在 k+1 處差值 >0，故 B 比 A 有更多 deadline 恰為 k+1 的作業。可選 x∈B\A 且 d_x=k+1。

- t≤k 時，N_t(A∪{x})=N_t(A)≤t。
- t≥k+1 時，整數差值給 N_t(A)+1≤N_t(B)≤t。

所以 A∪{x} 可行。連同遺傳性與空集合可行，得到 deadline matroid。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 完整選取表｜拒絕時給出違反前綴

依 penalty 遞減掃描，N 向量依序列出 t=1,…,7，接受者才更新 A。

| 考慮 | 候選 $N_1,\cdots,N_7$ | 決定 | 保住 penalty |
|---|---|---|---|
| $a_1$ | (0,0,0,1,1,1,1) | 接受 | 70 |
| $a_2$ | (0,1,1,2,2,2,2) | 接受 | 130 |
| $a_3$ | (0,1,1,3,3,3,3) | 接受 | 180 |
| $a_4$ | (0,1,2,4,4,4,4) | 接受 | 220 |
| $a_5$ | (1,2,3,5,5,5,5) | 拒絕，$N_4=5>4$ | 220 |
| $a_6$ | (0,1,2,5,5,5,5) | 拒絕，$N_4=5>4$ | 220 |
| $a_7$ | (0,1,2,4,4,5,5) | 接受 | 230 |

準時集合為 $\{a_1,a_2,a_3,a_4,a_7\}$，最小總罰款 $280-230=50$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 完整排程表｜選取順序不是執行順序

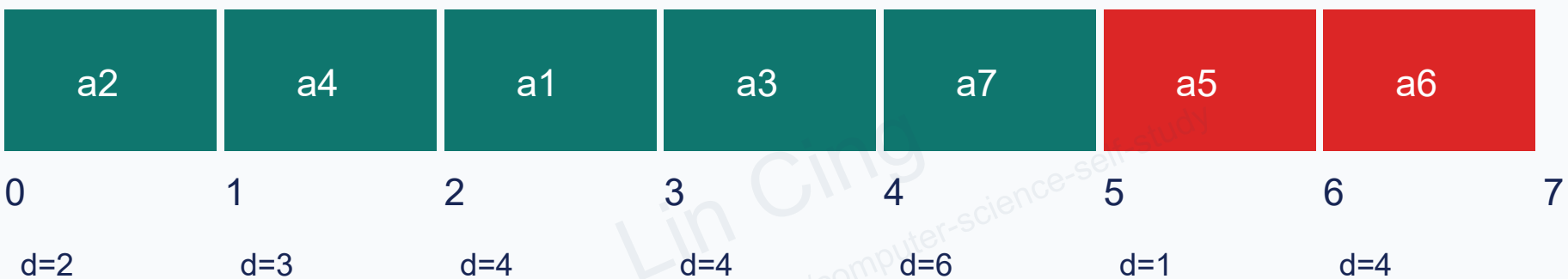
把接受的作業按 deadline 遞增排，再接上拒絕作業。相同 deadline 任意破同分。

| 執行格 | 作業 | 完成 C_i | $d_i$ | 狀態 | 罰款 |
|---|---|---|---|---|---|
| [0,1) | $a_2$ | 1 | 2 | 準時 | 0 |
| [1,2) | $a_4$ | 2 | 3 | 準時 | 0 |
| [2,3) | $a_1$ | 3 | 4 | 準時 | 0 |
| [3,4) | $a_3$ | 4 | 4 | 準時 | 0 |
| [4,5) | $a_7$ | 5 | 6 | 準時 | 0 |
| [5,6) | $a_5$ | 6 | 1 | 逾期 | 30 |
| [6,7) | $a_6$ | 7 | 4 | 逾期 | 20 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 完整排程表｜選取順序不是執行順序 （續）

先排準時集合，再排逾期作業




Σ saved = 230; Σ late penalty = 30 + 20 = 50

若改成不同處理時間、release time 或多處理器，須重新檢查可行集合的公理。本證明只涵蓋上述單位作業模型。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 16, Greedy Algorithms. 原始手寫掃描筆記《演算法_wjungle》貪婪法相關頁，涵蓋活動選擇、Huffman、擬陣與排程。 MIT 6.046J, Fall 2004, Lecture 13, Greedy Algorithms and Minimum Spanning Trees；Fall 2005, Lecture 16。兩份 MIT 講義用於貪婪法與 MST 證明框架，不作為 Huffman 原文來源。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study